

Joshua, Isaac, Oscar

## Type Invaders

*Preparation Phase:*

Features:

- Typing input to be registered and compared with text output
- Moving 2d area or body, able to act as a cannon
  - Have it autolock on target or through user input not registered by input box
- Shoot projectiles, whether it be area2d or sprite2d
- Have collisions
  - Projectiles capable of colliding with spawned enemies
  - Spawned enemies capable of disappearing of being collided
  - Game over state if enemies enter a colliding 2d area
- Multiple panels
  - Gameover, main menu, leaderboard

App Architecture Document:

- Discord
- Godot
- nodeJS
- SQLite
- Piskel
- Github
- Github desktop
- powershell

Research Log:

- Youtube video references:
- [Collision Shape Resource Mistake In Godot](#)
- [Create MAIN MENU for your Godot game](#)
- [Full Guide To 2D Collisions in Godot 4](#)
- [How to use CollisionShape2D in Godot Tutorial](#)
- [Godot 4 Projectiles Tutorial](#)
- [You're Not Too Dumb To Make Games](#)
- [Godot Typing Game Tutorial - Part 1 \(Handling Player Input\)](#)
- [Godot Typing Game Tutorial - Part 2 \(Colored Text and Enemy Spawns\)](#)
- [Godot Typing Game Tutorial - Part 3 \(Random Prompts and Difficulty Scaling\)](#)
- [Godot Typing Game Tutorial - Part 4 \(UI and Game Over\)](#)
- [How to make a Video Game - Godot Beginner Tutorial](#)
- [How to program in Godot - GDScript Tutorial](#)
- [Mastering Composition for Beginners in Godot](#)

- ChatGPT: to debug github push and commits, used to fix errors on the debugger, used to ask for random words for database, used to get a deeper understanding on collisions and difference between 2d character bodies, areas and sprites to implement movement

#### Style Guide / UI Design Document

- Cartoon style, light monochrome colors
- Pixel art for
  - Leaderboard
  - Projectile
  - Cannon
  - Main menu
  - Game over
  - Enemy
  - Background of gameplay loop
- Main menu
  - Start game button -> gameplay
  - Scores button -> leaderboard connected to a database
  - Exit button -> same functionality as pressing x on window
- Left and right arrow key input to move cannon 2d node
- Input box next to output label of current word.

#### *Main Features:*

##### Version control:

- github, push and pull through the github desktop app
- We all committed to the main branch onto `git@github.com:Jsalar68/3390_project3.git`
- Communication: via Discord server

##### Client-Side Data Model Classes:

- gd scripts attached to our 2d areas, sprites or control. The gd scripts are used to only modify the functionality of one node, which is later reused in our main game.gd script.

##### At least one pure server-side controller:

- Network.gd is the script that gets the http requests from the Node.js server
- Server located on Joshua's localhost with SQLite

##### Graphical UI/UX:

- The main game node contains multiple scenes like the Bullet.tscn, Main\_menu.tscn, Enemys.tscn, Game.tscn, Target.tscn, Score.tscn, Pause.tscn. Each of these scenes depending on their functionality contained reusable nodes with a 2d collision shape, labels, 2d markers, input labels, 2d areas, char body, Canvas Layer, etc.

### *Additional Features:*

#### Architecture Patterns:

- Model View Controller:
  - Model -> data structures
  - View -> user interface
  - Controller -> server logic (SQLite to Node.js to godot 4.6)

#### Persistent Data Storage:

- user scores along with their username, stored locally on a json file.
- Accessed through the Network.gd file.
- Scores and Usernames then updated on the label for the leaderboard panel

#### Design Patterns:

- Singleton: method applied when setting up everything and having our main functions be called through our `_ready()` function that runs once when the game starts and calls the functions that are important.
- Factory Method: `spawn_enemy()` used to create multiple enemy area2d nodes, that get called based on the `$SpawnTimer` timer and bullet nodes, created once correct word typed, and x position defined by the player user input keys of left or right arrow key.
- Iterator: used when comparing the input key and word matches the random word fetched from the database.

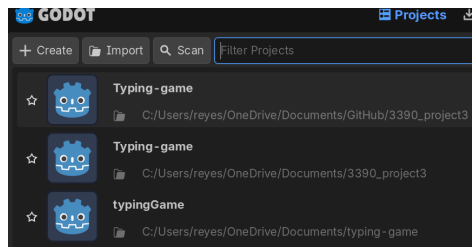
#### API testing:

- SQLite using get and post requests

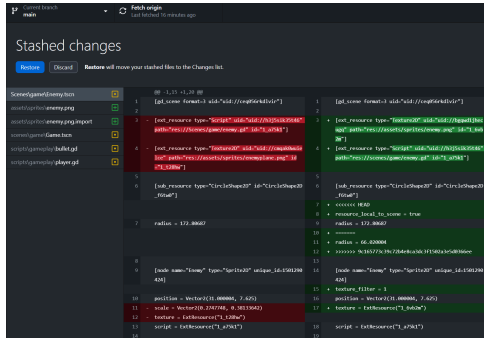
#### GitHub Desktop app:

"C:\Users\reyes\OneDrive\Documents\GitHub\3390\_project3"

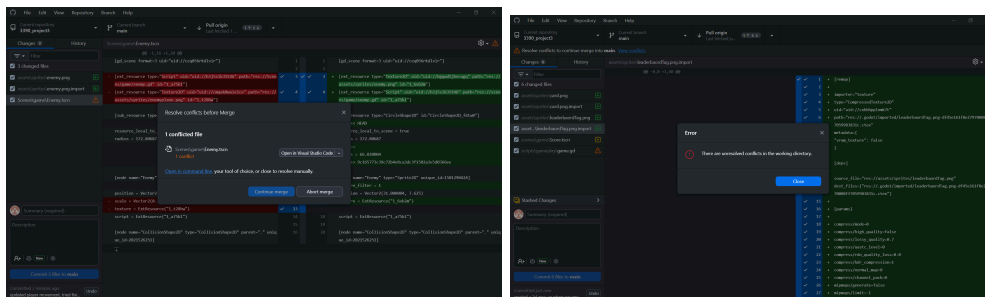
When trying to clone the repository we had issues finding where the clone was sent because godot likes to be inside the OneDrive folder under Documents, and we all had multiple Project3 and had the project named initially to typing game so we couldn't decipher which was which



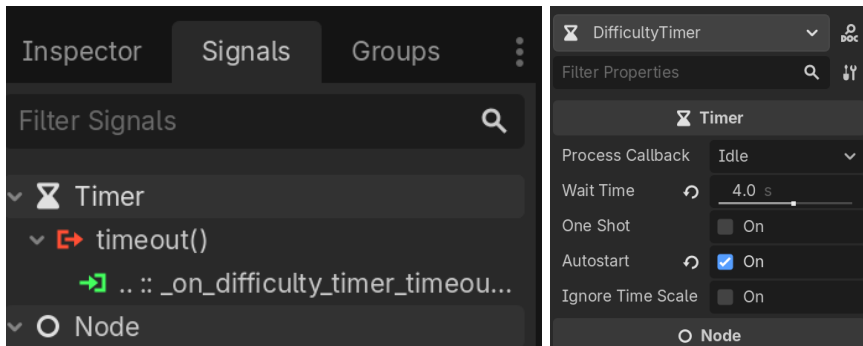
#### Merge conflicts and stashing on the github desktop app:



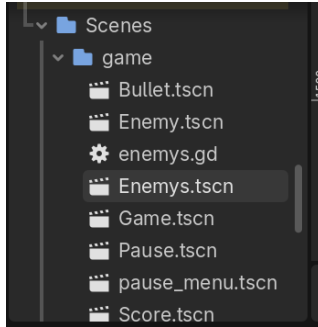
stashing our changes really helped with merge conflicts and we would look at what we could and couldn't merge, overall we had a lot of merging problems but it was just because everything depended or referenced a scene we updated or changed because it had a signal attached to the game.gd script



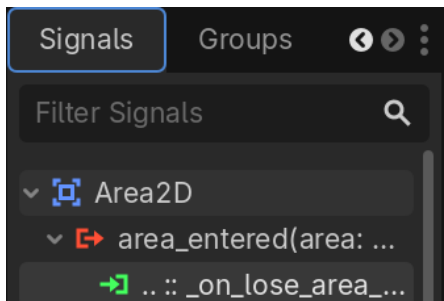
Whenever we had merge issues like these we would ask chatgpt or ask each other how to resolve the issue either through stashing or uncommitting



Often times we would have problems with signals when trying to attach them to the main game.gd script because once we turned it into a scene

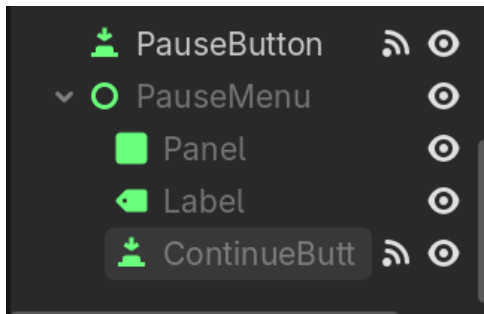


the signals would no longer be attached to the Game.tscn and the signal that was attached through a function for example `_on_button_pressed()` would no longer be useful inside the game.gd script.

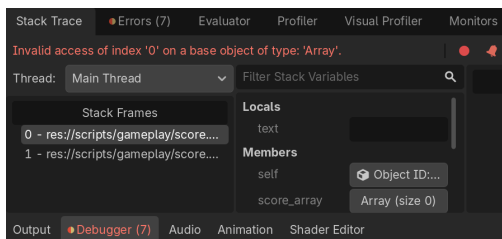


```
func _on_lose_area_area_entered(area: Area2D) -> void:
    print("FINAL SCORE: ", count)
    $wordInput.release_focus()
    var score_node := $game_Over/score_display/score_num
    score_node.text = str(count)
    print("After:", score_node.text)
    $game_Over.visible = true
```

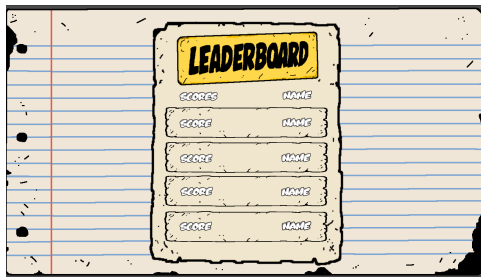
Collisions were a pain to implement, we didn't know previously that we could set up a signal to make the collisions work, we couldn't get collisions working with character bodies or sprites, so we had to use area2d to make collisions work



our pause menu couldn't be set up to be a scene because then we would have the ContinueButton signal emitted by the button pressed, and we needed the PauseButton and ContinueButton to be attached to the game.gd script.

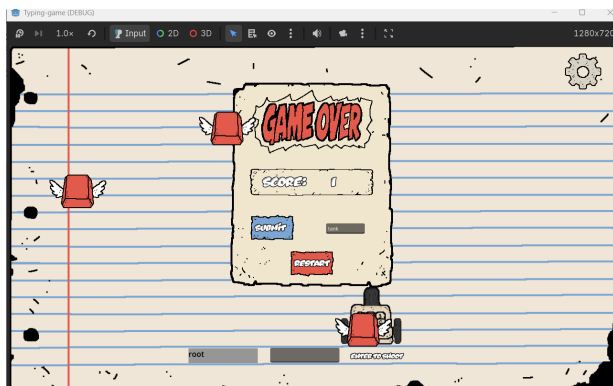


```
mainMe... 25 |>
Network... 26 |> $Scores/score_1.text = score_array[0]
pause.gd 27 |> $Name/name_1.text = name_array[0]
player.gd 28 |> $Scores/score_2.text = score_array[1]
score.gd 29 |> $Name/name_2.text = name_array[1]
30 |> $Scores/score_3.text = score_array[2]
31 |> $Name/name_3.text = name_array[2]
```



```
Server running on http://localhost:3000
SCORES ROUTE HIT
SCORES ROUTE HIT
SCORES ROUTE HIT
SCORES ROUTE HIT
SCORES ROUTE HIT
SCORES ROUTE HIT
SCORES ROUTE HIT
SCORES ROUTE HIT
BODY: { name: '', score: 1 }
NAME:
SCORE: 1
BODY: { name: 'tank', score: 1 }
NAME: tank
SCORE: 1
```

## SCREENSHOTS OF WORKING DATABASE:



## Backend

Consists of API GET and POST requests

Two-way communication between database and frontend.

Uses [node.js](https://nodejs.org/) as a bridge. Listens for requests and handles the communication

```
server.js  ✎ ✕ What's new
1  const express = require('express');
2  const Database = require('better-sqlite3');
3  const path = require('path');
4  console.log(__dirname);
5  const app = express();
6  app.use(express.json());
7
8  // connect to SQLite file
9  const db = new Database(path.join(__dirname, 'test.db'));
10
11 // create dummy table
12 db.prepare(`
13   CREATE TABLE IF NOT EXISTS logs (
14     id INTEGER PRIMARY KEY AUTOINCREMENT,
15     message TEXT
16   )
17 `).run();
18
19 db.prepare(`
20   CREATE TABLE IF NOT EXISTS words (
21     id INTEGER PRIMARY KEY AUTOINCREMENT,
22     word TEXT NOT NULL
23   )
24 `).run();
25
26 db.prepare(`
27   CREATE TABLE IF NOT EXISTS scores (
28     id INTEGER PRIMARY KEY AUTOINCREMENT,
29     name TEXT NOT NULL, score INT NOT NULL
30   )
31 `).run();
32
33
34 app.post('/score', (req, res) => {
35   console.log("BODY:", req.body);
36   const { name, score } = req.body;
37   console.log("NAME:", name);
38   console.log("SCORE:", score);
39   const scoreNum = Number(score);
40   db.prepare('INSERT INTO scores (name, score) VALUES (?,?)').run(name, scoreNum);
41   //console.log(info);
42   res.json({ status: "saved" });
43 });
44
45
46 app.get('/scores', (req, res) => {
47   console.log("SCORES ROUTE HIT");
48   const rows = db.prepare('SELECT * FROM scores ORDER BY score DESC').all();
49   res.json(rows);
50 });
51
52
53 // test route (to confirm Godot can reach server)
54 app.get('/ping', (req, res) => {
55   res.json({ status: "ok", message: "server reached" });
56 });
57
58 // route Godot will call
59 app.listen(3000, () => {
60   console.log("Server running on http://localhost:3000");
61 });
62
```

Script of [node.js](#)

Database is sqlite and consists of a single functional table, scores, tracking name and score.

```
sqlite> .tables
logs      scores
```